Sistema de Priorización de Atención: Colas, Árboles Balanceados, Heaps y Grafos para la Gestión de Solicitudes Urgentes

Johan Adrian Torres Robles, Esteban Mora Lopez, Miguel Angel Rodriguez Bohorquez
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{jotorresro, emoral, mrodriguezbo}@unal.edu.co

***Resumen*—Los centros de atención deben decidir en todo momento qué solicitud atender, clasificar su urgencia de forma consistente, ubicar y cancelar solicitudes puntuales, revertir errores de operación y, cuando una sede se satura, derivar a las personas a la sede con cupo más cercana. Este documento presenta el diseño, la implementación en Java y la evaluación de un *Sistema de Priorización de Atención* (SPA) que resuelve cada una de estas necesidades con la estructura de datos adecuada: colas enlazadas por nivel y un min-heap por plazo máximo como dos representaciones del mismo tipo abstracto de cola de pendientes; un árbol AVL como índice por identificador, comparado con un árbol binario de búsqueda ordinario; una pila para el historial y la operación de deshacer; Heap-Sort para listar los pendientes en orden de atención; un árbol de decisión recursivo para asignar el nivel de urgencia; y un grafo ponderado de sedes recorrido con BFS y Dijkstra para la derivación. Registrar, atender, buscar, cancelar y deshacer cuestan $O(\log n)$ en el peor caso. La implementación pasa 69 verificaciones automáticas, y los experimentos muestran que el AVL evita la degeneración del BST con identificadores consecutivos (altura 14 frente a 19 999 con $n = 20\,000$) y que la política por plazo elimina la inanición de las solicitudes menos urgentes bajo sobrecarga.**

***Index Terms*—cola de prioridad, heap binario, árbol AVL, árbol binario de búsqueda, Heap-Sort, árbol de decisión, grafos, Dijkstra, BFS, análisis amortizado**

I. Introducción

Los centros de atención que reciben solicitudes de servicio (salas de urgencias, mesas de ayuda técnica, módulos de atención al cliente) enfrentan un mismo reto operativo: las solicitudes no deben atenderse en el orden en que llegan, sino según su nivel de urgencia, y el personal necesita consultar o cancelar solicitudes específicas, revisar las atenciones realizadas y corregir errores sin reorganizar manualmente toda la fila. Cuando la institución tiene varias sedes, además, una sede saturada debería poder indicar a qué otra sede cercana enviar a las personas.

Resolver este problema de manera eficiente plantea varias dificultades computacionales distintas, y cada una pide una estructura diferente:

1. obtener en todo momento la solicitud que sigue sin ordenar repetidamente toda la colección, respetando el orden de llegada entre solicitudes equivalentes;
2. evitar que, bajo sobrecarga, las solicitudes menos urgentes esperen indefinidamente (inanición);
3. ubicar, cancelar o validar una solicitud por su identificador con un costo garantizado, aunque los identificadores lleguen en orden;
4. registrar las atenciones en el orden en que ocurren para poder consultarlas y revertir la más reciente;
5. asignar el nivel de urgencia de forma consistente, con un protocolo de preguntas y no por criterio libre de cada operador;
6. encontrar la sede con cupo más cercana dentro de una red de sedes conectadas por vías con distintos tiempos de traslado.

Este documento presenta el diseño, la implementación y la evaluación de un *Sistema de Priorización de Atención* (SPA) que resuelve cada dificultad con una estructura de datos justificada: colas sobre listas enlazadas y un min-heap como dos representaciones de la cola de pendientes; un árbol AVL como índice por identificador, comparado con un árbol binario de búsqueda (BST) ordinario; una pila para el historial; Heap-Sort para los reportes ordenados; un árbol de decisión para clasificar la urgencia; y un grafo ponderado de sedes con los algoritmos BFS y Dijkstra.

La contribución de este trabajo consiste en: (a) la formulación del problema, su alcance y sus requerimientos; (b) la selección y justificación técnica de cada estructura, con su análisis de complejidad; (c) una implementación funcional en Java, sin colecciones de la biblioteca estándar para las estructuras principales, verificada con 69 pruebas automáticas; y (d) una evaluación experimental que compara BST con AVL y las dos políticas de atención.

El resto del documento se organiza así: la Sección II presenta trabajo relacionado; la Sección III describe objetivos, alcance, requerimientos, diseño y estructuras de datos; la Sección IV reporta el estado de la implementación, las pruebas y los experimentos; la Sección V discute las decisiones y limitaciones; y la Sección VI resume el trabajo

y las líneas futuras.

II. Trabajo relacionado

Colas de prioridad. Atender elementos según su prioridad se modela con el tipo abstracto *cola de prioridad*, cuyas operaciones básicas son insertar y extraer el elemento más prioritario [1], [2]. El mismo comportamiento admite representaciones distintas, y es la representación la que fija el costo de cada operación: una lista ordenada extrae en $O(1)$ pero inserta en $O(n)$; una lista sin orden hace lo contrario; y el *heap* binario equilibra ambas operaciones en $O(\log n)$ [1], [4]. Cuando la prioridad solo toma un número pequeño y fijo de valores, basta una cola FIFO por valor, con ambas operaciones en $O(1)$. El SPA usa ambas representaciones detrás de una misma interfaz.

Triage y plazos máximos. Las escalas de triage hospitalario, como el sistema de Manchester [8], clasifican a cada paciente en cinco niveles mediante diagramas de preguntas encadenadas y asignan a cada nivel un tiempo máximo de espera (inmediato, 10, 60, 120 y 240 minutos). El protocolo de preguntas es, estructuralmente, un árbol de decisión. El tiempo máximo por nivel permite ordenar por *plazo más próximo*, idea clásica de planificación conocida como *earliest deadline first* [7], que evita que las tareas de baja prioridad esperen sin límite. El SPA toma ambas ideas como inspiración; no pretende ser un protocolo clínico.

Árboles de búsqueda balanceados. Un BST busca, inserta y elimina en $O(h)$, pero su altura depende del orden de inserción y llega a $n-1$ con claves ordenadas [1]. El árbol AVL [3] mantiene en cada nodo una diferencia de alturas entre subárboles de a lo sumo uno mediante rotaciones, lo que garantiza $h = O(\log n)$. Las tablas hash ofrecen búsqueda en $O(1)$ esperado, pero sin garantía de peor caso ni recorrido ordenado de las claves.

Heap-Sort y construcción de heaps. Heap-Sort [4] ordena en $\Theta(n \log n)$ en el peor caso con espacio auxiliar $O(1)$; la construcción ascendente de Floyd [5] arma el heap inicial en $O(n)$.

Caminos mínimos. En un grafo con pesos no negativos, el algoritmo de Dijkstra [6] calcula las distancias mínimas desde un origen; implementado con un heap binario cuesta $O((V+E) \log V)$. Cuando solo importa el número de aristas, el recorrido en anchura (BFS) lo resuelve en $O(V + E)$ [1].

Sistemas reales de mesa de ayuda y de triage hospitalario integran estas mismas piezas con persistencia, concurrencia e interfaces gráficas. Este proyecto no pretende replicar un sistema de producción, sino aislar y resolver correctamente su núcleo algorítmico.

III. Metodología

III-A. Objetivos

Objetivo general. Diseñar, implementar y evaluar un sistema que permita clasificar, registrar, priorizar, atender, cancelar, consultar y derivar solicitudes de atención de forma eficiente, aplicando para cada operación la estructura de datos apropiada y justificando su costo.

Objetivos específicos.

1. Implementar el tipo abstracto *cola de pendientes* con dos representaciones intercambiables: colas enlazadas por nivel, en $\Theta(1)$ por operación, y un min-heap por plazo máximo, en $O(\log n)$, que evite la inanición.
2. Implementar un índice AVL por identificador que garantice búsqueda, cancelación y validación en $O(\log n)$ en el peor caso, y compararlo experimentalmente con un BST ordinario.
3. Implementar una pila que registre el historial de atenciones y permita deshacer la más reciente.
4. Listar los pendientes en orden de atención con Heap-Sort, sin modificar la cola.
5. Implementar un árbol de decisión que asigne el nivel de urgencia mediante preguntas de sí o no.
6. Modelar la red de sedes como un grafo ponderado y derivar solicitudes a la sede con cupo más cercana con el algoritmo de Dijkstra.
7. Verificar la corrección con pruebas automáticas y medir el desempeño con experimentos reproducibles.

III-B. Alcance y limitaciones

El sistema registra solicitudes con identificador, nombre y nivel de urgencia (1, más urgente, a 5, menos urgente), asignado a mano o con el protocolo de preguntas; las atiende según la política elegida al iniciar; permite buscarlas, cancelarlas, deshacer la última atención, consultar el historial, listar pendientes y vencidas; y gestiona una red de seis sedes de ejemplo, cada una con su propia cola, índice e historial.

Quedan fuera del alcance: persistencia en archivos o bases de datos, interfaz gráfica, acceso concurrente, autenticación y cambio de urgencia de una solicitud ya registrada. El tiempo es un reloj lógico en minutos que avanza el operador. El protocolo de triage y la red de sedes son ejemplos ilustrativos fijados en el código. Se asume que los identificadores los proporciona quien registra, y que el número de niveles es la constante $k = 5$.

III-C. Requerimientos principales

- RF1: Registrar una solicitud con identificador único entre las pendientes, nombre no vacío y urgencia entre 1 y 5.
- RF2: Atender la siguiente solicitud según la política: por niveles (urgencia y luego llegada) o por plazo máximo.
- RF3: Buscar una solicitud pendiente por identificador.
- RF4: Cancelar una solicitud pendiente para que nunca sea atendida.
- RF5: Deshacer la atención más reciente, devolviendo la solicitud a la posición que tenía.
- RF6: Consultar el historial de atenciones.
- RF7: Listar los pendientes en el orden en que serán atendidos.
- RF8: Listar los pendientes ordenados por identificador.

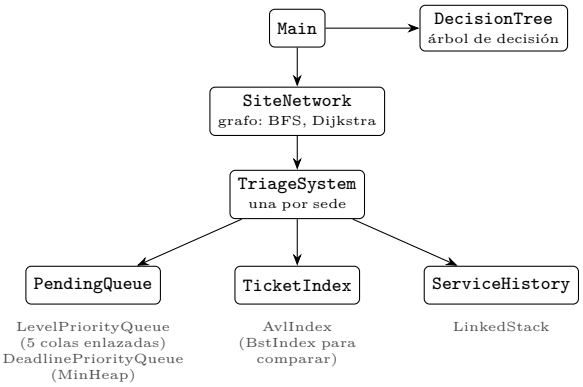


Figura 1. Componentes del sistema y estructura que resuelve cada responsabilidad. `MinHeap` se reutiliza en la cola por plazo, en Heap-Sort y en Dijkstra.

- RF9: Avanzar el reloj y listar las solicitudes con plazo vencido.
- RF10: Asignar la urgencia con un protocolo de preguntas.
- RF11: Indicar la sede con cupo más cercana y la ruta para llegar.
- RF12: Calcular la ruta mínima y el número de saltos entre sedes.

III-D. Diseño de la solución

La Fig. 1 muestra los componentes. `Main` ofrece el menú de consola y trabaja sobre la sede seleccionada de una `SiteNetwork`. Cada sede tiene un `TriageSystem`, fachada que coordina tres estructuras: la cola de pendientes, el índice y el historial. El `DecisionTree` es independiente de las sedes: solo produce el nivel de urgencia que luego se registra.

La Tabla I relaciona cada tema de la asignatura con el componente que lo aplica y la necesidad del problema que justifica usarlo. Ninguna estructura se incluye solo para aumentar su cantidad: cada una responde a una de las dificultades enumeradas en la introducción.

III-E. Estructuras de datos seleccionadas

En lo que sigue, n es el número de solicitudes pendientes de una sede, $k = 5$ el número de niveles, h la altura del árbol correspondiente, V el número de sedes y E el de vías.

III-E1. Colas sobre listas enlazadas: Cada `LinkedQueue` es una lista simplemente enlazada con referencias a la cabeza y al último nodo. La referencia al final permite encolar en $\Theta(1)$; sin ella habría que recorrer la lista. Se prefirió la lista sobre un arreglo circular porque el número de solicitudes por nivel no se conoce de antemano. Para deshacer se añadió `addFirst`, también $\Theta(1)$.

III-E2. Cola de pendientes por niveles: Es un arreglo de cinco colas; la cola u guarda, en orden de llegada, las solicitudes de urgencia u. Insertar es encolar en el nivel de la solicitud; atender es recorrer los niveles desde el más urgente y desencolar del primero no vacío (Algoritmo 1). Ambas operaciones cuestan $O(k) = \Theta(1)$ y el desempate por llegada no requiere comparaciones: lo garantiza el FIFO de cada cola. Su debilidad es que, bajo sobrecarga, el nivel 5 puede no atenderse nunca.

III-E3. Min-heap y cola por plazo máximo: Para evitar la inanición, la segunda política asigna a cada solicitud un plazo: minuto de llegada más el tiempo máximo de su nivel (0, 10, 60, 120 o 240 minutos) y atiende el plazo más próximo, desempatando por urgencia y luego por llegada. Como el plazo ya no toma cinco valores, la cola de niveles deja de servir y se usa un min-heap binario guardado en un arreglo con índices desde cero (hijos de i en $2i+1$ y $2i+2$). Sus invariantes son la *forma* (árbol completo, por lo que $h = \lfloor \log_2 n \rfloor$) y el *orden* (cada padre es menor o igual que sus hijos). Insertar coloca el elemento al final y lo sube con *sift-up*; extraer mueve el último a la raíz y lo baja con *sift-down* por el hijo de mayor prioridad. Ambas cuestan $O(\log n)$ con espacio auxiliar $O(1)$, pues son iterativas. El arreglo duplica su capacidad al llenarse, de modo que el costo de copiar es $\Theta(1)$ amortizado por inserción. El heap es genérico: recibe un `Comparator`, y el mismo código se reutiliza en Heap-Sort y en Dijkstra.

III-E4. Índice AVL por identificador: La cola responde "¿cuál sigue?", pero no "¿dónde está la solicitud X?". Sin un índice, buscar, cancelar y validar identificadores repetidos costaría $\Theta(n)$. El índice es un árbol AVL ordenado por identificador. Su invariante combina el orden BST (todas las claves del subárbol izquierdo son menores y las del derecho mayores) con el balance: $\mathrm{BF}(v) = h(v.izq) - h(v.der) \in \{-1, 0, 1\}$, con $h(\text{NULL}) = -1$. Tras cada inserción o eliminación recursiva se recalcula la altura y, si $|\mathrm{BF}| = 2$, se aplica la rotación del caso LL, RR, LR o RL (Algoritmo 2). Cada rotación cuesta $\Theta(1)$ y conserva el inorder, por lo que conserva el orden BST. El mínimo de nodos de un AVL de altura h cumple $N(h) = 1 + N(h-1) + N(h-2) \geq 2^{h/2}$, de donde $h = O(\log n)$ y las tres operaciones cuestan $O(\log n)$ en el peor caso. La eliminación con dos hijos copia el sucesor inorder y lo elimina del subárbol derecho.

Se descartó un BST ordinario porque los identificadores suelen ser consecutivos (A001, A002, ...): insertarlos en orden produce una cadena de altura $n-1$ y operaciones en $\Theta(n)$. El BST se conserva como `BstIndex` para demostrarlo experimentalmente. Se descartó una tabla hash porque no garantiza el peor caso y porque el AVL además entrega los pendientes ordenados por identificador con un recorrido inorder (RF8) en $\Theta(n)$.

III-E5. Cancelación perezosa e invariante de consistencia: Retirar un elemento arbitrario de un heap o de una cola exige primero encontrarlo, en $\Theta(n)$. Para cancelar en $O(\log n)$, el sistema elimina la solicitud del índice y solo la *marca* como cancelada; la cola la descarta cuando llega al frente. Cada solicitud cancelada se descarta una sola vez, así que el costo adicional es $\Theta(1)$ amortizado por cancelación. El sistema mantiene el invariante: *el*

Cuadro I
TEMAS DE LA ASIGNATURA, COMPONENTE QUE LOS APLICA Y NECESIDAD QUE JUSTIFICA SU USO

Tema	Componente	Necesidad del problema
TAD frente a representación	`PendingQueue`, `TicketIndex`	Cada interfaz tiene dos representaciones con distinto costo y el mismo comportamiento observable.
Listas enlazadas	`LinkedQueue`, `LinkedStack`, aristas del grafo	Tamaño desconocido de antemano; inserción en los extremos en $\Theta(1)$.
Colas (FIFO)	Niveles de `LevelPriorityQueue`; BFS	Desempate por orden de llegada; recorrido del grafo por niveles.
Pilas (LIFO)	`ServiceHistory`; inorder iterativo; reconstrucción de rutas	Deshacer la atención más reciente; invertir el camino de predecesores.
Arreglos dinámicos y análisis amortizado	Arreglo de `MinHeap`; eliminación perezosa	Crecer sin capacidad fija; descartar cancelados con costo amortizado $\Theta(1)$.
Recursión e inducción	`AvlIndex`, `DecisionTree`	Operaciones definidas sobre subárboles; pruebas de corrección por inducción estructural.
Recorridos de árboles	Inorder, preorder, por niveles	Pendientes por identificador, protocolo completo, altura del BST sin recursión.
BST	`BstIndex`	Línea base: muestra la degeneración con identificadores consecutivos.
AVL	`AvlIndex`	Búsqueda, cancelación y validación en $O(\log n)$ garantizado.
Heaps binarios	`MinHeap`, `DeadlinePriorityQueue`	Prioridad por plazo, con muchos valores posibles; cola de Dijkstra.
Heap-Sort	`MinHeap.heapSort`	Reporte de pendientes en orden de atención sin vaciar la cola.
Árboles de decisión (IA)	`DecisionTree`	Clasificar la urgencia con un protocolo consistente en $O(h)$ preguntas.
Grafos, BFS y Dijkstra	`SiteNetwork`	Derivar a la sede con cupo más cercana; saltos entre sedes.

índice contiene exactamente las solicitudes pendientes no canceladas, y todas ellas están en la cola; la cola puede tener además canceladas aún no descartadas. Por eso la cantidad de pendientes se lee del índice.

III-E6. Pila para el historial y deshacer: Cada atención se apila en $\Theta(1)$. Deshacer es exactamente LIFO: se desapila la última atención, se reinserta en el índice y se devuelve a la cola en la misma posición (Algoritmo 3). En la cola por niveles vuelve al *frente* de su nivel, porque cuando se atendió era el frente; en el heap basta reinsertarla, pues su clave no cambió. Si el identificador ya fue reutilizado, la operación se rechaza para no romper la unicidad.

III-E7. Heap-Sort para los reportes: Listar los pendientes en orden de atención (RF7) no debe vaciar la cola. Se copian del índice a un arreglo en $\Theta(n)$ y se ordenan con Heap-Sort in-place usando el comparador de la política activa: construcción ascendente de un max-heap en $O(n)$ y $n-1$ extracciones del máximo hacia el final del arreglo, cada una $O(\log n)$. Total $\Theta(n \log n)$ en el peor caso y espacio auxiliar $O(1)$ además de la copia.

III-E8. Árbol de decisión para clasificar la urgencia: Cada nodo interno es una pregunta de sí o no y cada hoja un nivel. El protocolo de ejemplo tiene 11 nodos, 6 hojas y altura 5. Clasificar sigue un único camino raíz-hoja, $O(h)$ preguntas sin importar el tamaño del árbol. Contar nodos, contar hojas y calcular la altura son recursivos, con caso base en el árbol vacío, y cuestan $\Theta(n)$ en tiempo y $\Theta(h)$ en espacio de pila. El protocolo se lista con un recorrido preorder. A diferencia de un BST, los nodos no guardan claves comparables: la posición indica la respuesta, no un orden.

III-E9. Grafo de sedes: La red es un grafo no dirigido y ponderado: cada vértice es una sede con capacidad y su propio `TriageSystem`, y cada arista una vía con su tiempo de traslado. Se representa con listas de adyacencia, de espacio $\Theta(V+E)$, preferidas sobre la matriz de adyacencia, de espacio $\Theta(V^2)$, porque cada sede tiene pocos vecinos. BFS, con una cola, calcula el menor número de saltos en $O(V+E)$. Dijkstra, con el mismo `MinHeap` y eliminación perezosa de entradas obsoletas, calcula los tiempos mínimos en $O((V+E)\log V)$; los pesos son positivos, condición que el algoritmo requiere y que el sistema valida. Para derivar (Algoritmo 4) se toma la sede no saturada con menor distancia y el camino se reconstruye siguiendo predecesores con una pila.

III-F. Resumen de complejidades

La Tabla II resume el costo de cada operación pública.

IV. RESULTADOS

IV-A. Estado de la implementación

La Tabla III resume el estado de cada funcionalidad. Las estructuras principales (colas, pila, heap, BST, AVL, árbol de decisión y grafo) son implementación propia; de la biblioteca estándar de Java solo se usan `ArrayList` para devolver listas, `Comparator` para definir órdenes y `Random` y `Scanner` en pruebas, experimentos y consola.

El programa `BasicTests` ejecuta 69 verificaciones automáticas, agrupadas en ocho bloques: escenario principal, casos borde (urgencia y textos inválidos, identificador

Cuadro II
Costo de las operaciones (n pendientes de la sede, h atenciones registradas)

Operación	Política por niveles	Política por plazo	Operaciones dominantes
Registrar (RF1)	$O(\log n)$	$O(\log n)$	Validar e insertar en el AVL; encolar $\Theta(1)$ o sift-up $O(\log n)$.
Atender (RF2)	$O(\log n)$	$O(\log n)$	Eliminar del AVL; descarte de canceladas $\Theta(1)$ amortizado.
Buscar (RF3)	$O(\log n)$	$O(\log n)$	Un camino raíz-hoja del AVL.
Cancelar (RF4)	$O(\log n)$	$O(\log n)$	Eliminar del AVL y marcar.
Deshacer (RF5)	$O(\log n)$	$O(\log n)$	Desapilar $\Theta(1)$; insertar en el AVL; restaurar.
Historial (RF6)	$\Theta(h)$	$\Theta(h)$	Recorrer la pila.
Pendientes en orden (RF7)	$\Theta(n \log n)$	$\Theta(n \log n)$	Inorder $\Theta(n)$ y Heap-Sort.
Pendientes por id (RF8)	$\Theta(n)$	$\Theta(n)$	Recorrido inorder del AVL.
Vencidas (RF9)	$\Theta(n)$	$\Theta(n)$	Recorrido inorder y comparación de plazos.
Clasificar (RF10)	$O(h_{\text{árbol}})$	$O(h_{\text{árbol}})$	Un camino raíz-hoja del árbol de decisión.
Derivar y ruta (RF11, RF12)	$O((V+E)\log V)$		Dijkstra con min-heap.
Saltos entre sedes (RF12)	$O(V+E)$		BFS con cola.

Algoritmo 1 Atender la siguiente solicitud (política por niveles)

```
1:  function ServeNext
2:      for u ← 1 to 5 do
3:          while L[u] no está vacía do
4:              t ← Dequeue(L[u])
5:              if t no está cancelada then
6:                  eliminar t.id del índice AVL
7:                  apilar t en el historial S
8:                  return t
9:              end if
10:         end while
11:     end for
12:     return NULL
13: end function
```

Cuadro III
Estado de las funcionalidades

Funcionalidad	Estado
Registrar, atender, buscar, cancelar (RF1–RF4)	Implementada
Deshacer la última atención (RF5)	Implementada
Historial y reportes ordenados (RF6–RF8)	Implementada
Reloj lógico y vencidas (RF9)	Implementada
Protocolo de triage (RF10)	Implementada
Derivación, rutas y saltos (RF11–RF12)	Implementada
Menú de consola	Implementada
Pruebas automáticas	Implementada
Experimentos de desempeño	Implementada
Cargar protocolo y red desde archivo	Pendiente
Cambiar la urgencia de una solicitud	Pendiente
Persistencia de datos	Fuera de alcance

Algoritmo 2 Rebalanceo de un nodo AVL

```
1:  function Rebalance(x)
2:      x.h ← 1 + máx(h(x.izq), h(x.der))
3:      if BF(x) > 1 then
4:          if BF(x.izq) < 0 then                ▷ caso LR
5:              x.izq ← RotarIzq(x.izq)
6:          end if
7:          return RotarDer(x)                   ▷ caso LL
8:      end if
9:      if BF(x) < −1 then
10:         if BF(x.der) > 0 then                ▷ caso RL
11:             x.der ← RotarDer(x.der)
12:         end if
13:         return RotarIzq(x)                   ▷ caso RR
14:     end if
15:     return x
16: end function
```

Algoritmo 3 Deshacer la atención más reciente

```
1:  function Undo
2:      if S está vacía then return NULL
3:      end if
4:      t ← Peek(S)
5:      if t.id está en el índice then
6:          error: id reutilizado
7:      end if
8:      Pop(S); insertar t en el índice AVL
9:      Restore(t)                               ▷ a su posición previa
10:     return t
11: end function
```

repetido, reutilización de identificadores atendidos o cancelados), deshacer, índices, heap y Heap-Sort, política por plazo, árbol de decisión y grafo. Cada verificación imprime `[OK]` o `[FALLA]` y el programa termina con código 1 si alguna falla. Entre ellas: tras 5000 inserciones y eliminaciones aleatorias el AVL conserva el orden, los

Algoritmo 4 Sede con cupo más cercana

```
1: function NearestAvailable(s)
2:     d[v] ← ∞ y p[v] ← null para todo v
3:     d[s] ← 0
4:     H ← min-heap con (s, 0)
5:     while H no está vacío do
6:         (u, ·) ← RemoveMin(H)
7:         if u ya fue procesado then continue
8:         end if
9:         for cada vía (u, v, w) do
10:            if d[u] + w < d[v] then
11:                d[v] ← d[u] + w; p[v] ← u
12:                insertar (v, d[v]) en H
13:            end if
14:        end for
15:    end while
16:    b ← sede v ≠ s no saturada con d[v] mínimo
17:    return b, d[b] y el camino armado con p y una pila
18: end function
```

Cuadro IV
Índice por identificador: altura y tiempo (ms) de n inserciones y n búsquedas

Ids	n	h BST	h AVL	ms BST	ms AVL
Ordenados	1000	999	9	12,3	1,1
Ordenados	5000	4999	12	142,0	2,8
Ordenados	20 000	19 999	14	3222,2	11,5
Aleatorios	1000	24	11	0,9	0,8
Aleatorios	5000	27	14	3,1	4,6
Aleatorios	20 000	38	16	20,6	26,9

factores de balance y las alturas guardadas; el BST con 1023 identificadores consecutivos queda con altura 1022 mientras el AVL queda perfecto, con altura 9; y en la red de ejemplo, con la sede Chapinero saturada, la derivación desde la sede Central elige la sede Sur (20 minutos). Las 69 verificaciones pasan.

IV-B. Evaluación experimental

El programa `Experiments` usa semillas fijas, por lo que los resultados son reproducibles; los tiempos corresponden a una ejecución en la JVM tras una ronda de calentamiento y varían entre equipos, pero las tendencias no.

BST frente a AVL. La Tabla IV mide n inserciones seguidas de n búsquedas exitosas. Con identificadores ordenados, el BST degenera en una cadena de altura $n-1$ y su tiempo crece de forma cuadrática: pasar de 5000 a 20 000 identificadores multiplica el tiempo por 23. El AVL mantiene altura logarítmica (14 con $n = 20\,000$, cerca de $\log_2 n \approx 14{,}3$) y es 280 veces más rápido. Con identificadores aleatorios el BST queda razonablemente balanceado por azar y es incluso algo más rápido que el AVL, que paga el costo de las rotaciones; la ventaja del AVL es la *garantía* de peor caso, no el caso promedio.

Niveles frente a heap. Con n inserciones seguidas de n extracciones y urgencias aleatorias, la cola por niveles tarda 1,8; 9,5 y 104,0 ms para $n = 10^4$, 10^5 y 10^6, y el heap 9,3; 70,6 y 845,0 ms. La cola por niveles crece linealmente, como corresponde a $\Theta(1)$ por operación, y el heap algo más rápido que linealmente, como corresponde a $O(\log n)$. El heap es unas ocho veces más lento, pero ambos costos son despreciables para el tamaño de una fila de atención.

Cuadro V
Sobrecarga de 480 minutos: urgencias 4 y 5

Política	Urg.	Atendidas	A tiempo	Espera máx.	Sin atender
Niveles	4	185	100 %	16	0
Niveles	5	29	55,2 %	359	90
Plazo	4	153	100 %	84	32
Plazo	5	68	100 %	205	51

Inanición bajo sobrecarga. Se simuló un turno de 480 minutos con 1,25 llegadas y una atención por minuto, de modo que la fila crece inevitablemente. La Tabla V muestra el resultado para las urgencias 4 y 5; los niveles 1 a 3 se atienden a tiempo en ambas políticas. Con la política por niveles, el nivel 5 casi no se atiende: 90 de 119 solicitudes quedan sin atender, la espera máxima llega a 359 minutos y solo el 55 % de las atendidas cumple su plazo de 240 minutos. Con la política por plazo, el retraso inevitable se reparte entre los niveles 3, 4 y 5, todas las solicitudes atendidas cumplen su plazo y la espera máxima del nivel 5 baja a 205 minutos.

V. Discusión

El diseño separa la responsabilidad de cada estructura en lugar de resolver todo con una sola. La cola de pendientes decide el orden, el índice ubica solicitudes, la pila recuerda las atenciones, el árbol de decisión clasifica y el grafo relaciona las sedes. Ninguna estructura resuelve bien todas las operaciones: el heap sabe cuál es la siguiente solicitud pero no dónde está una arbitraria, y el AVL sabe dónde está cada una pero no cuál sigue. Combinarlas obliga a mantener un invariante de consistencia, que el sistema protege concentrando todas las modificaciones en la fachada `TriageSystem` y verificándolo en las pruebas.

La separación entre tipo abstracto y representación permitió comparar alternativas sin cambiar al cliente. Las dos colas de pendientes implementan la misma interfaz y difieren en su costo y en su política; los dos índices implementan la misma interfaz y difieren en su garantía de peor caso. Los experimentos confirmaron los dos análisis teóricos: el BST degenera con identificadores consecutivos, y la cola por niveles, aunque más rápida, deja sin atender a los menos urgentes bajo sobrecarga.

La elección de política no es solo técnica. La política por niveles es la adecuada cuando la urgencia debe primar siempre, por ejemplo con pocos casos críticos y capacidad suficiente; la política por plazo es preferible cuando hay sobrecarga sostenida y se quiere acotar la espera de todos. Por eso el sistema permite elegirla al iniciar.

La cancelación perezosa intercambia memoria por tiempo: las solicitudes canceladas permanecen en la cola hasta llegar al frente, de modo que la memoria es $O(n + c)$ con c cancelaciones aún no descartadas, a cambio de cancelar en $O(\log n)$ en lugar de $\Theta(n)$.

Las principales limitaciones son: el protocolo de triage y la red de sedes están fijados en el código y son ilustrativos; cambiar la urgencia de una solicitud registrada exigiría una operación de disminución de clave en el heap, que requiere conocer la posición de cada elemento; los tiempos de la política por niveles dependen de que la urgencia tome solo cinco valores; y los datos existen solo durante la ejecución.

VI. Conclusiones

Este trabajo formuló el problema de priorizar, gestionar y derivar solicitudes de atención y lo resolvió con una estructura de datos justificada para cada necesidad: colas enlazadas por nivel y un min-heap por plazo como representaciones de la cola de pendientes, un árbol AVL como índice por identificador, una pila para deshacer, Heap-Sort para los reportes, un árbol de decisión para clasificar la urgencia y un grafo de sedes con BFS y Dijkstra para la derivación. Registrar, atender, buscar, cancelar y deshacer cuestan $O(\log n)$ en el peor caso.

La implementación cubre los doce requerimientos funcionales y pasa 69 verificaciones automáticas. Los experimentos mostraron que el AVL evita la degeneración del BST con identificadores consecutivos, con una altura de 14 frente a 19 999 y un tiempo 280 veces menor para 20 000 solicitudes, y que la política por plazo elimina la inanición del nivel menos urgente bajo sobrecarga, a cambio de un costo por operación mayor que el de la cola por niveles.

Como trabajo futuro se plantea: cargar el protocolo de triage y la red de sedes desde archivos; permitir cambiar la urgencia de una solicitud con una operación de disminución de clave sobre un heap indexado; migrar las pruebas a un marco como JUnit; y evaluar el sistema con datos de llegada más realistas.

Agradecimientos y declaración de ayudas

Los autores agradecen a la Universidad Nacional de Colombia y al equipo docente de la asignatura Estructuras de Datos. Se utilizó Claude (Anthropic), un asistente de inteligencia artificial, como apoyo en la implementación del código y en la redacción de este documento; el equipo revisó, probó y comprende todo el código y las decisiones de diseño presentadas.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.

[3] G. M. Adelson-Velsky y E. M. Landis, "An algorithm for the organization of information," *Soviet Mathematics Doklady*, vol. 3, pp. 1259–1263, 1962.

[4] J. W. J. Williams, "Algorithm 232: Heapsort," *Communications of the ACM*, vol. 7, no. 6, pp. 347–348, 1964.

[5] R. W. Floyd, "Algorithm 245: Treesort 3," *Communications of the ACM*, vol. 7, no. 12, p. 701, 1964.

[6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[7] C. L. Liu y J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[8] Manchester Triage Group, *Emergency Triage*, 3rd ed. Chichester, UK: Wiley-Blackwell, 2014.